

Q1

2025-04-07

Question 1

a)

The true values

We compute the exact values of $C(t)$ for t values from 1 to 100.

Monte Carlo Methods

We take $n=10^6$.

Transformation 1 For transformation 1, we use $u = \sin x$, then $x = \arcsin(u)$. When $x = 0$, then $u = 0$, and when $x = \pi/2$, then $u = 1$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t dx \stackrel{x=\arcsin(u)}{=} \int_0^1 u^t [\cos(\arcsin(u))^{-1} + [\cos(\arcsin(u))]^{t-1}] du \stackrel{\mathcal{U} \stackrel{d}{=} \mathcal{U}(0,1)}{=} \\ \mathbb{E} [u^t [\cos(\arcsin(u))^{-1} + [\cos(\arcsin(u))]^{t-1}]] \stackrel{MC}{\approx} \sum_{i=1}^N [u^t [\cos(\arcsin(u))^{-1} + [\cos(\arcsin(u))]^{t-1}]]$$

Using for loop to investigate the performance of integral across different values of t :

Transformation 2 For transformation 2, we use $u = \cos x$, then $x = \arccos(u)$. When $x = 0$, then $u = 0$, and when $x = \pi/2$, then $u = 1$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t dx \stackrel{x=\arccos(u)}{=} \int_0^1 u^t [\sin(\arccos(u))^{-1} + [\sin(\arccos(u))]^{t-1}] du \stackrel{\mathcal{U} \stackrel{d}{=} \mathcal{U}(0,1)}{=} \\ \mathbb{E} [u^t [\sin(\arccos(u))^{-1} + [\sin(\arccos(u))]^{t-1}]] \stackrel{MC}{\approx} \sum_{i=1}^N [u^t [\sin(\arccos(u))^{-1} + [\sin(\arccos(u))]^{t-1}]]$$

Using for loop to investigate the performance of integral across different values of t : According to the above results, transformation 1 and 2 have exactly the same mean and standard error.

Transformation 3 For transformation 3, we use $u = \frac{2}{\pi} * x$, then $x = \frac{\pi}{2}u$. When $x = 0$, then $u = 0$, and when $x = \pi/2$, then $u = 1$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t dx \stackrel{x=\frac{\pi}{2}u}{=} \int_0^1 [(\sin(\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t)] \frac{\pi}{2} du \stackrel{u \stackrel{d}{=} \mathcal{U}(0,1)}{=} \\ \mathbb{E} \left[\frac{\pi}{2} [(\sin(\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t)] \right] \stackrel{MC}{\approx} \sum_{i=1}^N \frac{\pi}{2} [(\sin(\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t)]$$

Using for loop to investigate the performance of integral across different values of t: The following table shows the comparison between different methods:

##	Transformation	Mean_Diff	Mean_Standard_Error
## 1	Transformation 1	-3.703208e-04	0.0026780224
## 2	Transformation 2	-3.703208e-04	0.0026780224
## 3	Transformation 3	-1.242511e-05	0.0004949451

According to the transformation methods above, transformation 3 has the smallest difference between estimated mean and actual mean, and also the smallest standard error. Thus we do the variance reduction based on transformation 3.

The antithetic variate reduction is not applicable since the function is not monotone. Then we try control variates method first.

1.Control variates method We want to find a random variable Y with mean 0 and a good constant c such that $\mathbb{E}(X + cY) \approx \theta$.

- Simulate $U_1, U_2, \dots, U_n \stackrel{d}{\sim} \mathcal{U}(0, 1)$.
- Simulate $\tilde{X} = (\frac{\pi}{2}u - \frac{(\frac{\pi}{2}u)^3}{6})^t + (1 - \frac{(\frac{\pi}{2}u)^2}{2})^t$, where $(\frac{\pi}{2}u - \frac{(\frac{\pi}{2}u)^3}{6})$ is the Taylor expansion of $\sin(\frac{\pi}{2}u)$, and $(1 - \frac{(\frac{\pi}{2}u)^2}{2})$ is the Taylor expansion of $\cos(\frac{\pi}{2}u)$.
- Simulate $Y = \tilde{X} - \mathbb{E}(\tilde{X})$.
- Calculate the $\text{Cov}(X, Y)$ and $\text{Var}(Y)$ with the samples to determine c^* , $c^* = -\frac{\text{Cov}(X, Y)}{\text{Var}(Y)}$
- Approximate θ with the mean of $X + c^*Y$ and standard errors with the standard errors of $X + c^*Y$.

Control Variate Method

Stratified Sampling We estimate:

$$h(U) = [(\sin(\frac{\pi}{2} \frac{U_j + j - 1}{n}))^t + (\cos(\frac{\pi}{2} \frac{U_j + j - 1}{n}))^t] \frac{\pi}{2}$$

Stratified Sampling + Control Variates

$$X = [(\sin(\frac{\pi}{2} \frac{U_j + j - 1}{n}))^t + (\cos(\frac{\pi}{2} \frac{U_j + j - 1}{n}))^t] \frac{\pi}{2}$$

$$\tilde{X} = [(\frac{\pi}{2} \frac{U_j + j - 1}{n} - \frac{\pi}{2} \frac{(\frac{U_j + j - 1}{n})^3}{6})^t + (1 - \frac{(\frac{\pi}{2} \frac{U_j + j - 1}{n})^2}{2})^t] \frac{\pi}{2}$$

$$Y = \tilde{X} - \mathbb{E}(\tilde{X})$$

Calculate the $\text{Cov}(X, Y)$ and $\text{Var}(Y)$ with the samples to determine c^* , $c^* = -\frac{\text{Cov}(X, Y)}{\text{Var}(Y)}$.

Approximate θ with the mean of $X + c^*Y$ and standard errors with the standard errors of $X + c^*Y$.

```
# Compare SS + CV with pure Monte Carlo without any variance reduction
(abs(Diff_3) - abs(Diff_cvss))[1:10]
```

Compare all variance reduction methods

```
## [1] 6.811730e-05 0.000000e+00 1.591679e-04 2.628563e-05 8.088983e-06
## [6] 1.334485e-04 3.564456e-04 1.293248e-04 3.460293e-04 7.889249e-07
```

```
(round((Std_Error_3 - SE_cvss)/Std_Error_3, digits = 2))[1:10]
```

```
## [1] 0.60 0.00 0.49 0.63 0.65 0.65 0.64 0.62 0.60 0.58
```

```
# Compare CV with pure Monte Carlo without any variance reduction
(abs(Diff_3) - abs(Diff_cv3))[1:10]
```

```
## [1] 0 0 0 0 0 0 0 0 0 0
```

```
(round((Std_Error_3 - SE_cv3)/Std_Error_3, digits = 2))[1:10]
```

```
## [1] 0.60 0.00 0.49 0.63 0.65 0.65 0.64 0.62 0.60 0.58
```

This table represents the differences between two variance reduction methods (SS+CV, only CV) with Monte Carlo Methods for $t = 1, 2, \dots, 10$.

```
compare_df <- data.frame(
  t = 1:10,
  diff_SSCV = (abs(Diff_3) - abs(Diff_cvss))[1:10],
  sd_improved_SSCV = (round((Std_Error_3 - SE_cvss)/Std_Error_3, digits = 2))[1:10],
  diff_CV = (abs(Diff_3) - abs(Diff_cv3))[1:10],
  sd_improved_CV = (round((Std_Error_3 - SE_cv3)/Std_Error_3, digits = 2))[1:10]
)
compare_df
```

	t	diff_SSCV	sd_improved_SSCV	diff_CV	sd_improved_CV
## 1	1	6.811730e-05	0.60	0	0.60
## 2	2	0.000000e+00	0.00	0	0.00
## 3	3	1.591679e-04	0.49	0	0.49
## 4	4	2.628563e-05	0.63	0	0.63
## 5	5	8.088983e-06	0.65	0	0.65
## 6	6	1.334485e-04	0.65	0	0.65
## 7	7	3.564456e-04	0.64	0	0.64
## 8	8	1.293248e-04	0.62	0	0.62
## 9	9	3.460293e-04	0.60	0	0.60
## 10	10	7.889249e-07	0.58	0	0.58

(b)

Calculate the true value of the double integral

```
set.seed(1)
f <- function(x, y) {
  r <- sqrt(x^2 + y^2)
  ifelse(r == 0, 1, sin(r) / r) # Handle the removable singularity at (0,0)
}

result <- integrate(function(y) {
  sapply(y, function(yi) {
    integrate(function(x) f(x, yi), -1, 1)$value
  })
}, -1, 1)

# Print the result
true_integral_1b <- result$value
true_integral_1b
```

```
## [1] 3.575761
```

Pure Monte Carlo Methods

Since the equation is symmetric, we can compute:

$$\int_{-1}^1 \int_{-1}^1 \frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}} dx dy = 4 \int_0^1 \int_0^1 \frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}} dx dy \stackrel{(X,Y) \text{ i.i.d. } \mathcal{U}(0,1)}{=} 4 \mathbb{E}\left[\frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}}\right] \stackrel{MC}{\approx} \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{x_i^2 + y_i^2})}{\sqrt{x_i^2 + y_i^2}} dx dy,$$

where $X_1, \dots, X_N \stackrel{d}{\sim} \mathcal{U}(0,1)$, $Y_1, \dots, Y_N \stackrel{d}{\sim} \mathcal{U}(0,1)$, and the samples are independent.

1. Antithetic Variate Reduction We need to calculate

$$\frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{(x^2 + y^2)})}{\sqrt{x^2 + y^2}} = \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{((1-x)^2 + (1-y)^2)})}{\sqrt{(1-x)^2 + (1-y)^2}},$$

where $X_1, \dots, X_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, $Y_1, \dots, Y_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, and the samples are independent.

```
improve_av <- (Std_Error_mc - Std_Error_av)/Std_Error_mc
improve_av
```

```
## [1] 0.7587932
```

The antithetic variate reduction reduces the standard error by 75.8793%.

2.Stratified Sampling Since $1 - U_i \stackrel{d}{=} U_i$, we can compute

$$h(U) = \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{(\frac{U_i+i-1}{n})^2 + (\frac{U_j+j-1}{n})^2})}{\sqrt{(\frac{U_i+i-1}{n})^2 + (\frac{U_j+j-1}{n})^2}},$$

where $X_1, \dots, X_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, $Y_1, \dots, Y_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, and the samples are independent.

```
improve_SS <- (EstIntegral_mc - Mu_SS)/EstIntegral_mc
improve_SS
```

```
## [1] -0.001564384
```

Stratified Sampling + Antithetic Variate reduction To combine the stratified sampling with antithetic variate reduction, we calculate the mean of the following to estimations:

$$h(U) = \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{(\frac{U_i+i-1}{n})^2 + (\frac{U_j+j-1}{n})^2})}{\sqrt{(\frac{U_i+i-1}{n})^2 + (\frac{U_j+j-1}{n})^2}},$$

and

$$h(1 - U) = \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{(\frac{i-U_i}{n})^2 + (\frac{j-U_j}{n})^2})}{\sqrt{(\frac{i-U_i}{n})^2 + (\frac{j-U_j}{n})^2}}.$$

```
improve_SS <- (EstIntegral_mc - Mu_avss)/EstIntegral_mc
improve_SS
```

```
## [1] -0.001564385
```

The following table shows the comparison between different methods:

##	Method	Mean	Diff	Standar_error
## 1	Ture Value_1b	3.575761	NA	NA
## 2	MC_1b	3.575741	2.05941642512641e-05	0.000821174490111747
## 3	AV_1b	3.575570	0.000191293603578391	0.000198072897562495
## 4	SS_1b	3.581334	-0.00557323728978432	0.00115390418221003
## 5	AVSS_1b	3.581335	-0.00557323939834875	0.00115390416790357

Therefore, the antithetic variate reduction can reduce the variance by 75.8793%, and the difference between estimated mean and the true mean is the smallest. Therefore, the antithetic variate reduction is the best method to improve the accuracy.